

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

(Университет ИТМО)

Факультет прикладной информатики

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии

О Т Ч Е Т

Тема задания: «ЗАПРОСЫ НА ВЫБОРКУ И МОДИФИКАЦИЮ ДАННЫХ.
ПРЕДСТАВЛЕНИЯ. РАБОТА С ИНДЕКСАМИ»

Обучающийся: Данилова Айаана Васильевна, К3241

Санкт-Петербург, 2026

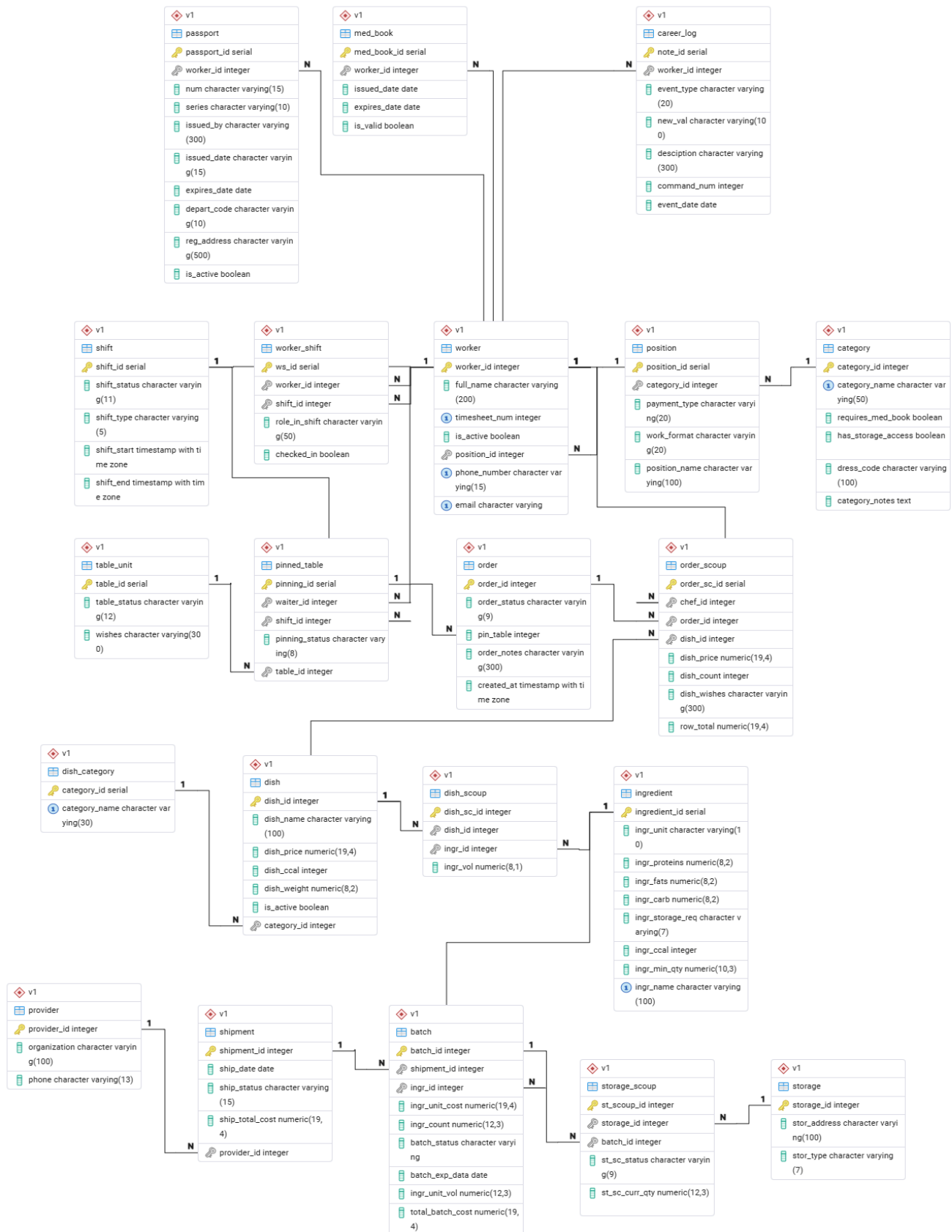
Цель работы

Овладеть практическими навыками создания представлений и запросов на выборку данных к базе данных PostgreSQL, использования подзапросов при модификации данных и индексов.

Практическое задание:

1. Создать запросы и представления на выборку данных к базе данных PostgreSQL (согласно индивидуальному заданию лабораторной работы №2, часть 2 и 3).
2. Составить 3 запроса на модификацию данных (INSERT, UPDATE, DELETE) **с использованием подзапросов**.
3. Изучить графическое представление запросов и просмотреть историю запросов.
4. Создать простой и составной индексы для двух произвольных запросов и сравнить время выполнения запросов без индексов и с индексами. Для получения плана запроса использовать команду EXPLAIN.

Схема базы данных (ЛР)



Краткое описание:

В ресторане пока предусмотрено 3 типа сотрудников, которые соответственно относятся каждый к одной из трех категорий: Зал, Кухня, Администрация. Паспортная информация всех работников хранится со статусом документа, чтобы отслеживать действующий. Должности из категории “Кухня” обязательно иметь мед. книжку, что фиксировано в сущности категории с помощью булевого флага. Предусмотрена кадровая история. Непосредственно основные типы сотрудников:

- официант, за которым закрепляются столы(внешний ключ для сущности закрепленный стол) на определенной смене(тоже внешний ключ, важно на этапе открытия смены и распределения столами между официантами на смене по факту присутствия)
- администратор, ответственный за смену
- повар, готовящий блюда из заказа(внешний ключ в составе заказа)

Ресторан не специализируется на какой-то определенной кухне и имеет обычное и универсальное меню, разделенное по категориям, то есть каждое блюдо входит в какую-то одну категорию.

Ингредиенты поставляются партиями компаниями поставщиками и хранятся на складах.

Бд подходит для одного филиала, но это не мешает пользоваться ею и крупным конгломератам с несколькими филиалами, так как для каждого филиала бд можно поднять на одном из серверов или ноды кластера сервера.

Выполнение

1. Запросы к бд

1.1. Вывести данные официанта, принявшего заказы на максимальную сумму за истекший месяц

Решение: Через цепочку джойнов от заказов к составу заказа, закрепленным столам и сменам официантов выбираем заказы за последний месяц, группируем суммы по каждому официанту, затем оконной функцией rank() ранжируем по убыванию и возвращаем строку с рангом 1.

```
with WaiterSales as (  
  select ws.worker_id, sum(os.row_total) as total_sales  
  from   (select * from v1.order where created_at >=  
current_date::time with time zone - interval '1 month')  
  as o  
  join v1.order_scoop os on os.order_id = o.order_id  
  join v1.pinned_table pt on pt.pinning_id = o.pinning_id  
  join v1.worker_shift ws on ws.ws_id = pt.ws_id and  
ws.position_in_shift = 10  
  group by ws.worker_id  
)  
  
select worker_id, total_sales  
from (  
  select *, rank() over (order by total_sales desc) as rnk  
  from WaiterSales  
) as t  
where rnk = 1;
```

Результат:

	worker_id integer	total_sales numeric
1	4447	17969.8376

✓ Запрос выполнен успешно. Общее время выполнения: 104 мс. обработано строк: 1. ✕

1.2. Рассчитать премию каждого официанта за последние 10 дней (5% от стоимости каждого заказа).

Решение: Аналогично запросу 1.1 собираем суммы заказов по каждому официанту за последние 10 дней, используя right join с pinned_table чтобы включить официантов даже без заказов, и применяем coalesce для замены null на 0 при умножении суммы на 0.05.

```
with WaiterSales as (  
  select ws.worker_id, sum(os.row_total) as total_sales  
  from (select * from v1.order where created_at >= current_date -  
        interval '10 days')  
  as o  
  join v1.order_scoop os on os.order_id = o.order_id  
  right join v1.pinned_table pt on pt.pinning_id = o.pinning_id  
  join v1.worker_shift ws on ws.ws_id = pt.ws_id and  
  ws.position_in_shift = 10  
  group by ws.worker_id  
)  
  
select worker_id, coalesce(total_sales * 0.05, 0) as bonus  
from WaiterSales;
```

Результат:

	worker_id integer	bonus numeric
1	4447	898.491880
2	4322	546.851960
3	4550	382.539100
4	4783	353.221360
5	4202	337.830785
6	4255	254.692325
7	4509	253.510275
8	4309	252.131830
9	4385	234.051225
10	4241	231.706900
11	4417	191.050985
12	4350	0
13	4799	0
14	4287	0
15	4688	0
16	4225	0
17	4395	0
18	4432	

✓ Запрос выполнен успешно. Общее время выполнения: 89 мс. обработано строк: 29. ✕

1.3. Подсчитать, сколько ингредиентов содержит каждое блюдо.

Решение: Джойним таблицу блюд dish с таблицей состава dish_scoop и группируем по идентификатору и названию блюда, считая количество строк через count(*).

```
select d.dish_id, d.dish_name, count(*) as ingr_count
from v1.dish d
join v1.dish_scoop ds on ds.dish_id = d.dish_id
group by d.dish_id, d.dish_name
```

Результат:

Showing rows: 1 to 7 / Page No: 1 of 1

	dish_id [PK] integer	dish_name character varying (100)	ingr_count bigint
1	3	Борщ с говядиной	2
2	5	Паста Карбонара	4
3	4	Стейк Рибай	2
4	2	Цезарь с курицей	4
5	7	Двойной Эспрессо	1
6	6	Чизкейк	3
7	1	Английский	1

Всего строк: 7 Запрос завершен 00:00:00.118 CRLF Ln 4, Col 32

✓ Запрос выполнен успешно. Общее время выполнения: 118 мс. обработано строк: 7. ✕

1.4. Вывести название блюда, содержащее максимальное число ингредиентов.

Решение: В CTE считаем количество ингредиентов для каждого блюда, затем во внешнем запросе оконной функцией rank() ранжируем по убыванию и возвращаем только строки с рангом 1 — их может быть несколько при равенстве максимума.

```
with ingrCount as (
select d.dish_id, d.dish_name, count(ds.ingr_id) as ingr_count
from v1.dish d
join v1.dish_scoop ds on ds.dish_id = d.dish_id
group by d.dish_id
)
select dish_name
from (select *, rank() over (order by ingr_count desc) as rnk from
ingrCount) where rnk = 1
```

Результат:

	dish_name character varying (100) 🔒
1	Цезарь с курицей
2	Паста Карбонара

✓ Запрос выполнен успешно. Общее время выполнения: 121 мс. обработано строк: 2. ✕

Всего строк: 2 | Запрос завершен 00:00:00.121 | CRLF | Ln 1, Col 1

1.5. Какой повар может приготовить максимальное число видов блюд?

Решение: В CTE считаем количество уникальных блюд по каждому повару на смене из order_scoop за последний год, сразу ранжируем оконной функцией и фильтруем по рангу 1.

```
with cook_variety as (  
    select  
        w.worker_id,  
        w.full_name,  
        count(distinct os.dish_id) as unique_dishes_count,  
        rank() over (order by count (distinct os.dish_id)  
desc) as rnk  
  
    from v1.order o  
        join v1.order_scoop os on o.order_id = os.order_id  
        join v1.worker_shift ws on os.ws_id = ws.ws_id  
        join v1.worker w on w.worker_id = ws.worker_id  
        where o.created_at >= current_date - interval '1 year'  
        group by w.worker_id  
  
)  
select *  
from cook_variety cv  
where rnk = 1;
```


Результат:

	worker_id [PK] integer	full_name character varying (200)	unique_dishes bigint
1	4335	Новиков Фёдор	6
2	4367	Ефимова Лариса	6
3	4370	Степанов Роман	6
4	4377	Фёдоров Антон	6
5	4446	Соколов Денис	6
Всего строк: 600		Запрос завершен 00:00:05.282	

1.6. Сколько закреплено столов за каждым из официантов за сегодняшний день?

Решение: Джойним pinned_table, worker_shift и worker, фильтруем по текущей дате через приведение pinned_at::date = current_date, группируем по работнику и считаем количество закреплений через count(pt.pinning_id), сортируя по убыванию.

```
select
    w.worker_id,
    w.full_name,
    count(pt.pinning_id) as tables_today
from v1.pinned_table pt
join v1.worker_shift ws on ws.ws_id = pt.ws_id
join v1.worker w on w.worker_id = ws.worker_id
where pt.pinned_at::date = current_date
group by w.worker_id, w.full_name
order by tables_today desc;
```

Результат:

	worker_id [PK] integer	full_name character varying (200)	tables_today bigint
1	4711	Зайцева Наталья	2
2	4310	Козлова Ксения	1
3	4350	Макарова Ксения	1
4	4771	Соколов Дмитрий	1
5	4799	Михайлова Виктория	1
Всего строк: 5		Запрос завершен 00:00:00.160	

1.7. Какой из ингредиентов используется в максимальном количестве блюд?

Решение: В CTE считаем для каждого ингредиента количество уникальных блюд из dish_scoop и сразу присваиваем ранг оконной функцией, затем джойним с ingredient для получения названия и возвращаем только ингредиенты с рангом 1.

```
with ingr_usage as (  
  select  
    ds.ingr_id,  
    count(distinct ds.dish_id) as dish_count,  
    rank() over (order by count(distinct ds.dish_id) desc) as  
rnk  
  from v1.dish_scoop ds  
  group by ds.ingr_id  
)  
select  
  i.ingredient_id,  
  i.ingr_name,  
  iu.dish_count  
from ingr_usage iu  
join v1.ingredient i on i.ingredient_id = iu.ingr_id  
where iu.rnk = 1;
```

Результат:

	ingredient_id [PK] integer	ingr_name character varying (100)	dish_count bigint
1	8	Говядина вырезка	2
2	9	Картофель	2
3	11	Сливки 33%	2
4	14	Мука пшеничная	2
5	1	Яйцо куриное	2
Всего строк: 7		Запрос завершен 00:00:00.111	

2. Представления

2.1. Для расчета стоимости ингредиентов для заданного блюда

```
create or replace view v1.vw_dish_ingredient_cost as
with latest_price as (
    select distinct on (b.ingr_id)
        b.ingr_id,
        b.ingr_unit_cost,
        b.ingr_unit_vol,
        b.batch_exp_data
    from v1.batch b
    join v1.shipment s      on s.shipment_id = b.shipment_id
    join v1.storage_scoup ss on ss.batch_id   = b.batch_id
    where s.ship_status    = 'accepted'
        and ss.st_sc_status = 'available'
        and b.batch_exp_data > current_date
    order by b.ingr_id, b.batch_exp_data asc
)
select
    d.dish_id,
    d.dish_name,
    i.ingredient_id,
    i.ingr_name,
    i.ingr_unit                                as unit,
    ds.ingr_vol                                as
qty_in_dish,
    lp.ingr_unit_cost                          as
price_per_unit,
    round(ds.ingr_vol * lp.ingr_unit_cost, 4)  as
ingredient_cost
from v1.dish_scoup ds
join v1.dish      d on d.dish_id      = ds.dish_id
join v1.ingredient i on i.ingredient_id = ds.ingr_id
left join latest_price lp on lp.ingr_id = ds.ingr_id;
```

Применение:

```
select *
from v1.vw_dish_ingredient_cost
where dish_id = 2;
```

	dish_id integer	dish_name character varying (100)	ingredient_id integer	ingr_name character varying (100)	unit v1.measure_unit	qty_in_dish numeric (12,3)	price_per_ numeric (1
1	2	Цезарь с курицей	4	Салат Романо	кг	0.060	
2	2	Цезарь с курицей	5	Куриное филе	кг	0.120	
3	2	Цезарь с курицей	6	Сыр Пармезан	кг	0.015	
4	2	Цезарь с курицей	7	Соус Цезарь	л	0.040	

✓ Запрос выполнен успешно. Общее время выполнения: 80 мс. обработано строк: 4. ✕

Всего строк: 4 Запрос завершен 00:00:00.080 CRLF Ln 3, Col 19

Содержимое представления:

1 `select * from v1.vw_dish_ingredient_cost`

Результат Сообщения Уведомления

+

📄

▼

📋

▼

🗑️

🗑️

⬇️

📈

SQL

Showing rows: 1 to 19 Page No: 1 of 1 ⏪ ⏩ ⏴ ⏵

	dish_id integer	dish_name character varying (100)	ingredient_id integer	ingr_name character varying (100)	unit v1.measure_unit	qty_in_dish numeric (12,3)
1	1	Английский завтрак	1	Яйцо куриное	шт	3.000
2	1	Английский завтрак	2	Бекон	кг	0.060
3	1	Английский завтрак	3	Томаты Черри	кг	0.050
4	2	Цезарь с курицей	5	Куриное филе	кг	0.120
5	2	Цезарь с курицей	4	Салат Романо	кг	0.060
6	2	Цезарь с курицей	7	Соус Цезарь	л	0.040
7	2	Цезарь с курицей	6	Сыр Пармезан	кг	0.015
8	3	Борщ с говядиной	8	Говядина вырезка	кг	0.080
9	3	Борщ с говядиной	9	Картофель	кг	0.150
10	4	Стейк Рибай	8	Говядина вырезка	кг	0.350
11	4	Стейк Рибай	9	Картофель	кг	0.100
12	5	Паста Карбонара	14	Мука пшеничная	кг	0.100
13	5	Паста Карбонара	2	Бекон	кг	0.050
14	5	Паста Карбонара	11	Сливки 33%	л	0.060
15	5	Паста Карбонара	6	Сыр Пармезан	кг	0.020

Всего строк: 19 Запрос завершен 00:00:00.224 CRLF Ln 1, Col 36

2.2. Для всех поваров количество приготовленных блюд по каждому блюду за определенную дату

```
create or replace view v1.vw_chef_dishes_per_date as
select
    o.created_at::date                as cook_date,
    w.worker_id                     as chef_id,
    w.full_name                     as chef_name,
    d.dish_id,
    d.dish_name,
    sum(os.dish_count)              as portions_cooked,
    count(distinct os.order_id)     as orders_count
from v1.order_scoop os
join v1.order o on o.order_id = os.order_id
join v1.worker_shift ws on ws.ws_id = os.ws_id
join v1.worker w on w.worker_id = ws.worker_id
join v1.dish d on d.dish_id = os.dish_id
where os.status in ('ready', 'served')
group by
    o.created_at::date,
    w.worker_id,
    w.full_name,
    d.dish_id,
    d.dish_name;
```

Применение:

```
select chef_name, sum(portions_cooked) as total_today
from v1.vw_chef_dishes_per_date
where cook_date = '2026-04-25'
group by chef_name
order by total_today desc;
```

	chef_name character varying (200) 🔒	total_today numeric 🔒
1	Фёдоров Максим	21
2	Ефимов Сергей	18
3	Лебедев Илья	15
4	Попов Борис	15
5	Ефимова Вероника	15
6	Степанов Роман	14
7	Волкова Татьяна	12
8	Михайлов Алексей	10
Всего строк: 98		Запрос завершен 00:00:01.009

Содержимое представления:

1

select * from v1.vw_chef_dishes_per_date

Результат

Сообщения

Уведомления

≡

📄

▼

📋

▼

🗑️

🔍

📥

⬇️

📈

SQL

Showir

	cook_date date 🔒	chef_id integer 🔒	chef_name character varying (200) 🔒	dish_id integer 🔒	dish_name character varying (100) 🔒	portions_cooked bigint 🔒	orders_count bigint 🔒
1	2024-04-25	4202	Ефимова Ирина	2	Цезарь с курицей	1	1
2	2024-04-25	4202	Ефимова Ирина	3	Борщ с говядиной	3	1
3	2024-04-25	4202	Ефимова Ирина	5	Паста Карбонара	1	1
4	2024-04-25	4202	Ефимова Ирина	6	Чизкейк	3	1
5	2024-04-25	4206	Попов Леонид	3	Борщ с говядиной	2	1
6	2024-04-25	4206	Попов Леонид	4	Стейк Рибай	2	1
7	2024-04-25	4206	Попов Леонид	5	Паста Карбонара	3	1
8	2024-04-25	4332	Зайцев Борис	3	Борщ с говядиной	2	1
9	2024-04-25	4332	Зайцев Борис	4	Стейк Рибай	2	1
10	2024-04-25	4332	Зайцев Борис	7	Двойной Эспрессо	1	1
11	2024-04-25	4374	Волков Александр	3	Борщ с говядиной	3	1
12	2024-04-25	4374	Волков Александр	4	Стейк Рибай	2	1
13	2024-04-25	4374	Волков Александр	6	Чизкейк	3	1
14	2024-04-25	4410	Смирнова Мария	2	Цезарь с курицей	2	1
15	2024-04-25	4410	Смирнова Мария	3	Борщ с говядиной	3	1
16	2024-04-25	4410	Смирнова Мария	4	Стейк Рибай	3	1
17	2024-04-25	4410	Смирнова Мария	5	Паста Карбонара	2	1
18	2024-04-25	4410	Смирнова Мария	6	Чизкейк	2	1
19	2024-04-25	4410	Смирнова Мария	7	Двойной Эспрессо	2	1

Всего строк: 60321

Запрос завершен 00:00:02.170

3. Запросы на модификацию данных (INSERT, UPDATE, DELETE) с использованием подзапросов

3.1. Повышение цен для блюд стоимостью дешевле 500 руб

```
update v1.dish
set dish_price = dish_price * 1.10
where category_id in (
    select category_id
    from v1.dish
    group by category_id
    having avg(dish_price) < 500
);
```

Таблица с блюдами и ценами до :

	category_id integer	avg_before numeric (10,2)
1	6	181.50
2	1	544.50
3	3	471.90
4	5	423.50
5	4	875.00
6	2	580.00

После:

	category_id integer	avg_after numeric (10,2)
1	6	199.65
2	1	544.50
3	3	519.09
4	5	465.85
5	4	875.00
6	2	580.00

✓ Запрос выполнен успешно. Общее время выполнения: 239 мс. обработано строк: 6. ✕

Всего строк: 6

Запрос выполнен: 00:00:00.041

CPLE | L5 SQL

3.2. Удаление будущих смен

```
delete from v1.worker_shift
where shift_id in (
    select s.shift_id
    from v1.shift s
    where s.shift_start >= current_date
)
and worker_id in (
    select worker_id
    from v1.med_book
    where is_active = false
);
```

До:

	count_to_delete bigint
1	500

После:

Результат	Сообщения	Уведомления
DELETE 500		
Запрос завершён успешно, время выполнения: 58 мс.		

3.3. Запись о найме в кадровой истории в качестве повара


```

insert into v1.career_log (worker_id, note_text)
select worker_id, 'staff member of kitchen department'
from v1.worker
where position_id in (
    select position_id
    from v1."position" p
    join v1.category c on p.category_id = c.category_id
    where c.category_name = 'kitchen'
);

```

До:

1199	2999	4750	COMMENDATION	Благодарность	Поощрение за высокие показатели работы
1200	3000	4336	COMMENDATION	Благодарность	Поощрение за высокие показатели работы
Всего строк: 1200 Запрос завершен 00:00:00.343 CRLF Ln 1, Col 28					

После:

1418	3218	4317	DEPARTMENT_ASSIGN	Kitchen	staff member of kitchen department
1419	3219	4318	DEPARTMENT_ASSIGN	Kitchen	staff member of kitchen department
1420	3220	4477	DEPARTMENT_ASSIGN	Kitchen	staff member of kitchen department
1421	3221	4479	DEPARTMENT_ASSIGN	Kitchen	staff member of kitchen department
1422	3222	4480	DEPARTMENT_ASSIGN	Kitchen	staff member of kitchen department
1423	3223	4717	DEPARTMENT_ASSIGN	Kitchen	staff member of kitchen department
1424	3224	4718	DEPARTMENT_ASSIGN	Kitchen	staff member of kitchen department
Всего строк: 1424 Запрос завершен 00:00:00.175 CRLF Ln 16, Col 1					

4. Индексы

В рамках пункта выше был придуман вариант модификации удаления неактивных сотрудников из смены, но вскоре стало понятно что это бесполезное действие, так как в продакшене как такового удаления записи нет, есть мягкое удаление через маркер статуса, и перезапись, но добить поставленную задачу все равно хотелось. Пришлось руками удалять матрешку, то есть зависимые в качестве внешних ключей друг друга записей закрепленных столов, заказ, состава заказа, субд на каждом шаге удаления делала полный перебор, чтобы найти непосредственно эти зависимости:

```
36 );  
37  
38 COMMIT;
```

Всего строк: 481277	Ожидание завершения запроса... 00:38:36.549
---------------------	---

В PostgreSQL индексы на PRIMARY KEY создаются автоматически, а на FOREIGN KEY - нет, потому были добавлены:

```
create index idx_order_scoup_order_id on v1.order_scoup(order_id);  
create index idx_order_pinning_id on v1."order"(pinning_id);  
create index idx_pinned_table_ws_id on v1.pinned_table(ws_id);  
create index idx_worker_shift_worker_id on v1.worker_shift(worker_id);
```

CREATE INDEX

Запрос завершён успешно, время выполнения: 1 мин 21 сек.

Результат:

Результат Сообщения Уведомления

COMMIT

Запрос завершён успешно, время выполнения: 44 сек 88 мс.

Добавленные индексы помогли сократить выполнение запроса до 44 секунд.

Удаление индексов:

```
drop index if exists v1.idx_order_scoup_order_id;  
drop index if exists v1.idx_order_pinning_id;  
drop index if exists v1.idx_pinned_table_ws_id;
```

```
drop index if exists v1.idx_worker_shift_worker_id;

analyze v1.order_scoup;
analyze v1."order";
analyze v1.pinned_table;
analyze v1.worker_shift;
```

Но зафиксировать план не удалось, но так как модификация уже была применена, то ниже представлен второй кейс.

Сравнение на SELECT запросе

Запрос анализирует эффективность поваров за август 2025 года: для каждого повара на каждой смене он считает, сколько порций каждого блюда было приготовлено, какую выручку это принесло и сколько заказов было обслужено, но только по закрытым заказам, поданным блюдам дороже 300 рублей, у которых сумма строки превышает 500. На основе этих агрегатов по каждому повару строятся три ранга: место внутри смены по количеству порций, место внутри смены по выручке и место среди всех поваров за день по порциям. Результат отсортирован по дате смены и рангу по порциям.

Исходный запрос:

```
select
  s.shift_id,
  s.shift_start::date           as work_date,
  s.shift_type                  as shift_type,
  w.full_name                   as chef_name,
  d.dish_name,
  d.dish_price,

  sum(os.dish_count)            as total_portions,
  sum(os.row_total)             as total_revenue,
  avg(os.dish_price_snapshot)   as avg_price,
  count(distinct o.order_id)    as orders_count,

  rank() over (
    partition by s.shift_id
```

```

        order by sum(os.dish_count) desc
    )
        as rank_by_portions_in_shift,

    rank() over (
        partition by s.shift_id
        order by sum(os.row_total) desc
    )
        as rank_by_revenue_in_shift,

    rank() over (
        partition by s.shift_start::date
        order by sum(os.dish_count) desc
    )
        as rank_by_portions_in_day

from v1.worker_shift ws

join v1.shift s          on s.shift_id          = ws.shift_id
    and s.shift_start >= '2025-08-01'
    and s.shift_start <  '2025-09-01'

join v1.pinned_table pt on pt.ws_id            = ws.ws_id
    and pt.pinning_status = 'serviced'

join v1."order" o        on o.pinning_id = pt.pinning_id
    and o.order_status = 'closed'

join v1.order_scoop os   on os.order_id        = o.order_id
    and os.status = 'served'

join v1.worker w         on w.worker_id       = ws.worker_id
    and w.is_active = true

join v1.dish d           on d.dish_id         = os.dish_id
    and d.dish_price > 300

where os.row_total > 500

group by
    s.shift_id,
    s.shift_start,
    s.shift_type,

```

```
w.worker_id,
w.full_name,
d.dish_id,
d.dish_name,
d.dish_price
```

```
order by
  s.shift_start,
  rank_by_portions_in_shift,
  w.full_name;
```

План с анализом:

#	Node	Rows	Loops
		Actual	
1.	→ Sort (rows=10 loops=1)	10	1
2.	→ Window Aggregate (rows=10 loops=1)	10	1
3.	→ Incremental Sort (rows=10 loops=1)	10	1
4.	→ Window Aggregate (rows=10 loops=1)	10	1
5.	→ Sort (rows=10 loops=1)	10	1
6.	→ Window Aggregate (rows=10 loops=1)	10	1
7.	→ Sort (rows=10 loops=1)	10	1
8.	→ Aggregate (rows=10 loops=1)	10	1
9.	→ Incremental Sort (rows=256 loops=1)	256	1
10.	→ Nested Loop Inner Join (rows=256 loops=1)	256	1
11.	→ Nested Loop Inner Join (rows=256 loops=1) Join Filter: (pt.pinning_id = o.pinning_id)	256	1
12.	→ Nested Loop Inner Join (rows=2 loops=1)	2	1
13.	→ Merge Inner Join (rows=2 loops=1)	2	1
14.	→ Sort (rows=13 loops=1)	13	1
15.	→ Nested Loop Inner Join (rows=19 loops=1)	19	1
16.	→ Seq Scan on pinned_table as pt (rows=19 loops=1) Фильтр: (pinning_status = 'serviced':v1.pinning_status) Rows Removed by Filter: 42	19	1
17.	→ Index Scan using worker_shift_pkey on worker_shift as ws (rows=1 ... Index Cond: (ws_id = pt.ws_id)	1	19

18.	→ Sort (rows=62 loops=1)	62	1
19.	→ Seq Scan on shift as s (rows=62 loops=1) Фильтр: ((shift_start >= '2025-08-01 00:00:00+03':timestamp with time zone) AND (shift_end < '2025-08-01 00:00:00+03':timestamp with time zone)) Rows Removed by Filter: 1484	62	1
20.	→ Index Scan using worker_pkey on worker as w (rows=1 loops=2) Фильтр: is_active Index Cond: (worker_id = ws.worker_id) Rows Removed by Filter: 0	1	2
21.	→ Hash Inner Join (rows=7114 loops=2) Hash Cond: (os.order_id = o.order_id)	7114	2
22.	→ Seq Scan on order_scoop as os (rows=21852 loops=2) Фильтр: ((row_total > '500':numeric) AND (status = 'served':v1.order_item_status)) Rows Removed by Filter: 99894	21852	2
23.	→ Hash (rows=6658 loops=1) Buckets: 8192 Batches: 1 Memory Usage: 325 kB	6658	1
24.	→ Seq Scan on order as o (rows=6658 loops=1) Фильтр: (order_status = 'closed':v1.order_status) Rows Removed by Filter: 13633	6658	1
25.	→ Memoize (rows=1 loops=256) Buckets: Batches: Memory Usage: 1 kB	1	256
26.	→ Index Scan using pk_dish on dish as d (rows=1 loops=5) Фильтр: (dish_price > '300':numeric) Index Cond: (dish_id = os.dish_id) Rows Removed by Filter: 0	1	5
о строк: 1 Запрос завершен 00:00:01.422		CRLF	Ln 15, Col 1

Результат:

Результат План выполнения Сообщения Уведомления

	shift_id integer	work_date date	shift_type v1.shift_type	chef_name character varying (200)	dish_name character varying (100)	dish_price numeric (19,4)	total_portions bigint	total_revenue numeric	avg_price numeric	orders_count bigint	rank_by_pt bigint
1	3998	2025-08-10	Day	Ефимова Ирина	Цезарь с курицей	580.0000	53	30740.0000	580.0000000000000000	26	
2	3998	2025-08-10	Day	Ефимова Ирина	Стейк Рибай	1200.0000	51	61200.0000	1200.0000000000000000	26	
3	3998	2025-08-10	Day	Ефимова Ирина	Борщ с говядиной	519.0900	50	19500.0000	390.0000000000000000	21	
4	3998	2025-08-10	Day	Ефимова Ирина	Чизкейк	465.8500	50	17500.0000	350.0000000000000000	20	
5	3998	2025-08-10	Day	Ефимова Ирина	Паста Карбонара	550.0000	46	25300.0000	550.0000000000000000	22	
6	3999	2025-08-10	Night	Ефимова Ирина	Стейк Рибай	1200.0000	80	96000.0000	1200.0000000000000000	33	
7	3999	2025-08-10	Night	Ефимова Ирина	Цезарь с курицей	580.0000	78	45240.0000	580.0000000000000000	41	
8	3999	2025-08-10	Night	Ефимова Ирина	Борщ с говядиной	519.0900	63	24570.0000	390.0000000000000000	23	
9	3999	2025-08-10	Night	Ефимова Ирина	Паста Карбонара	550.0000	53	29150.0000	550.0000000000000000	29	
10	3999	2025-08-10	Night	Ефимова Ирина	Чизкейк	465.8500	37	12950.0000	350.0000000000000000	15	
Всего строк: 10 Запрос завершен 00:00:01.372											
										CRLF	Ln 15, Col 1

Применим индексы:

```
create index
idx_pt_pinning_status on v1.pinned_table(pinning_status)

create index
idx_pt_ws_id on v1.pinned_table(ws_id)
where pinning_status = 'serviced';

create index
idx_os_order_id on v1.order_scoop(order_id);

create index
idx_os_dish_id on v1.order_scoop(dish_id)
where status = 'served';
```

```
create index
idx_o_pinning_id on v1.order(pinning_id)
where order_status = 'closed';

create index
idx_s_shift_start on v1.shift(shift_start);

create index
idx_ws_worker_shift_id on v1.worker_shift(worker_id, shift_id);

create index
idx_w_full_name on v1.worker(full_name)
where is_active = true;

analyze v1.order;
analyze v1.order_scoup;
analyze v1.worker_shift;
analyze v1.shift;
analyze v1.pinned_table;
analyze v1.worker;
```

Время выполнения запроса после добавления и применения индексов:

Результат Сообщения План выполнения X Уведомления

Запрос выполнен успешно. Общее время выполнения: 81 мс.
обработано строк: 10.

Время сократилось до 81 мс против исходных ~1.4 секунды.

План с анализом:

#	Node	Rows	Loops
		Actual	
1.	→ Sort (rows=10 loops=1)	10	1
2.	→ Window Aggregate (rows=10 loops=1)	10	1
3.	→ Incremental Sort (rows=10 loops=1)	10	1
4.	→ Window Aggregate (rows=10 loops=1)	10	1
5.	→ Sort (rows=10 loops=1)	10	1
6.	→ Window Aggregate (rows=10 loops=1)	10	1
7.	→ Sort (rows=10 loops=1)	10	1
8.	→ Aggregate (rows=10 loops=1)	10	1
9.	→ Incremental Sort (rows=256 loops=1)	256	1
10.	→ Nested Loop Inner Join (rows=256 loops=1)	256	1
11.	→ Nested Loop Inner Join (rows=256 loops=1)	256	1
12.	→ Nested Loop Inner Join (rows=222 loops=1)	222	1
13.	→ Nested Loop Inner Join (rows=2 loops=1)	2	1
14.	→ Merge Inner Join (rows=2 loops=1)	2	1
15.	→ Sort (rows=13 loops=1)	13	1
16.	→ Merge Inner Join (rows=19 loops=1)	19	1
17.	→ Index Scan using worker_shift_pkey on worker_shift as ws (rows=1...	1343	1
18.	→ Sort (rows=19 loops=1)	19	1
19.	→ Seq Scan on pinned_table as pt (rows=19 loops=1) Фильтр: (pinning_status = 'serviced'::v1.pinning_status) Rows Removed by Filter: 42	19	1
20.	→ Sort (rows=62 loops=1)	62	1
21.	→ Bitmap Heap Scan on shift as s (rows=62 loops=1) Recheck Cond: ((shift_start >= '2025-08-01 00:00:00+03':timestamp with time z Heap Blocks: exact=2	62	1
22.	→ Bitmap Index Scan using idx_s_shift_start (rows=62 loops=1) Index Cond: ((shift_start >= '2025-08-01 00:00:00+03':timestamp with time	62	1
23.	→ Index Scan using worker_pkey on worker as w (rows=1 loops=2) Фильтр: is_active Index Cond: (worker_id = ws.worker_id) Rows Removed by Filter: 0	1	2
24.	→ Bitmap Heap Scan on order as o (rows=111 loops=2) Recheck Cond: ((pinning_id = pt.pinning_id) AND (order_status = 'closed'::v1.order_status)) Heap Blocks: exact=220	111	2
25.	→ Bitmap Index Scan using idx_o_pinning_id (rows=111 loops=2) Index Cond: (pinning_id = pt.pinning_id)	111	2
26.	→ Index Scan using idx_os_order_id on order_scoop as os (rows=1 loops=222) Фильтр: ((row_total > '500':numeric) AND (status = 'served'::v1.order_item_status)) Index Cond: (order_id = o.order_id) Rows Removed by Filter: 5	1	222
27.	→ Memoize (rows=1 loops=256) Buckets: Batches: Memory Usage: 1 kB	1	256
28.	→ Index Scan using pk_dish on dish as d (rows=1 loops=5) Фильтр: (dish_price > '300':numeric) Index Cond: (dish_id = os.dish_id) Rows Removed by Filter: 0	1	5

В сравнении с планом до можно заметить, что:

- Узел 22) Bitmap Index Scan using idx_s_shift_start (rows=62)
Вместо полного прохода по shift PostgreSQL использует индекс idx_s_shift_start и сразу получает только 62 смены за август 2025. Это замена бывшему Seq Scan (узел 19 первого плана, rows=62, Rows Removed by Filter: 1484) — раньше сканировались все 1546 смен, теперь только нужные 62.
- Узел 25) Bitmap Index Scan using idx_o_pinning_id (rows=111)
Появился частичный индекс idx_o_pinning_id on v1.order(pinning_id) where order_status = 'closed'. PostgreSQL идёт напрямую к 111

нужным строкам с закрытыми заказами через Bitmap Heap Scan (узел 24), вместо прежнего Seq Scan по 6658 строкам с последующей фильтрацией 13633 лишних.

- Узел 26) Index Scan using idx_os_order_id on order_scoup (rows=1, loops=222)

Вместо двух полных сканирований таблицы order_scoup (21852 строк $\times$ 2 = 43704 обращений) теперь используется индекс idx_os_order_id. PostgreSQL делает 222 точечных обращения по одной строке. Rows Removed by Filter: 5 почти нет потерь. Это переход от $O(N)$ к $O(\log N) \times$ количество совпадений.

- Узел 17) Index Scan using worker_shift_pkey (rows=1343)

Соединение с worker_shift теперь идёт через первичный ключ. В первом плане это тоже было (узел 17), но rows=1, loops=19, т.е. 19 точечных обращений. Во втором плане rows=1343 — но это нормально, потому что сам путь к этому узлу сократился: планировщик выбрал другую последовательность джойнов.

Выводы

Результатом работы стало глубокое понимание взаимодействия между логической структурой БД и физическим уровнем выполнения запросов. Были освоены методы профилирования через EXPLAIN ANALYZE, что позволило диагностировать узкие места (например, избыточные сканирования или неэффективные соединения). Применение индексов и нормализации данных было реализовано не только с точки зрения синтаксиса, но и с позиции оптимизации ресурсов СУБД.